

# Experimental plan to measure inference efficiency and component usage using vLLM and CuTe-DSL

## Background: GPU memory hierarchy and tile addressing

Modern GPUs use a hierarchy of memory types. Registers and on-chip **shared memory** (SMEM) offer **extremely high bandwidth** (SMEM  $\approx 20$  TB/s but limited to  $\sim 220$  kB per SMX), while **high-bandwidth memory** (HBM) provides tens of gigabytes of capacity at lower bandwidth. To achieve high performance, GPU kernels use *tiling*: data from the large HBM is copied into small shared-memory tiles that can be processed efficiently. NVIDIA's CuTe-DSL exposes this hierarchy. The base pointer for a tile is computed from the row and column offsets according to the formula

$$\text{ptrs} = \text{base} + \text{nrows}[0, \text{None}] \times \text{nrow\_stride} + \text{cols}[\text{None}, 0] \times \text{col\_stride}$$

Here `nrows` and `cols` are vectors of the row and column indices in the tile and the `[0, None]` and `[None, 0]` broadcasting (the "None trick") generates a 2D grid of addresses in HBM <sup>1</sup>. Choosing appropriate `nrow_stride` and `col_stride` ensures that threads in a warp access consecutive addresses, minimising *bank conflicts* and *non-coalesced loads*. CuTe's layout primitives allow one to describe row-major or column-major layouts and to compose them into tiled layouts <sup>1</sup>.

## vLLM metrics for inference monitoring

The vLLM engine exposes a rich set of **server-level** and **request-level** metrics via a Prometheus-compatible `/metrics` endpoint. Key metrics relevant to your study include:

| Metric                                                                                                      | Description                                  | Importance                                                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------|
| <code>vllm:num_requests_running</code> (Gauge)                                                              | Number of concurrent requests                | overall engine load <sup>2</sup>                                             |
| <code>vllm:kv_cache_usage_perc</code> (Gauge)                                                               | Fraction of used KV-cache blocks (0-1)       | measures KV-cache pressure and swapping <sup>2</sup>                         |
| <code>vllm:prefix_cache_queries</code> / <code>vllm:prefix_cache_hits</code> (Counters)                     | Number of prefix-cache lookups and hits      | compute hit rate for prefix reuse <sup>3</sup>                               |
| <code>vllm:request_prefill_time_seconds</code> / <code>vllm:request_decode_time_seconds</code> (Histograms) | Time spent in the pre-fill and decode phases | measure compute time spent in attention/MLP vs token generation <sup>4</sup> |

| Metric                                                                                          | Description                                               | Importance                          |
|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------|-------------------------------------|
| <code>vllm:time_to_first_token_seconds</code> and <code>vllm:inter_token_latency_seconds</code> | Time-to-first-token (TTFT) and inter-token latency (TPOT) | user-perceived latency <sup>5</sup> |

The [vLLM metrics design document](#) explains that server-level metrics help diagnose request-level metrics and that these metrics can be logged or scraped by Prometheus <sup>6</sup>.

**Enabling metrics and logs.** When running the vLLM engine (as in the Forest of Audits repository), do not set `--disable-log-stats` so that the engine periodically prints statistics (throughput, KV-cache usage and prefix-cache hit rate) to the console. You can adjust log frequency with the `VLLM_LOG_STATS_INTERVAL` environment variable <sup>7</sup>. Setting `VLLM_LOGGING_LEVEL=DEBUG` enables more verbose logging <sup>8</sup>. The `llm.get_metrics()` method can be used programmatically to retrieve engine metrics after a batch completes <sup>9</sup>.

## Measuring component usage at the vLLM inference-engine level

1. **Run the vLLM engine in evaluation harnesses.** Use the *Forest of Audits* repository to run the `frontier-evals` tasks. In each harness (e.g., OpenAI's `opencode` harness, Codex harness), launch vLLM with metrics logging enabled:

```
python -m vllm.entrypoints.openai.api_server \
  --model <hf-model-id> \
  --max-model-len 4096 \
  --port 8000 \
  --disable-log-stats=False # enable periodic stats
```

Set `max-model-len` according to the expected prompt length; oversizing the KV cache wastes VRAM and reduces throughput <sup>10</sup>.

1. **Collect per-request metrics.** When generating completions via the OpenAI-compatible API or directly through `LLM.generate()`, record the number of prompt tokens, generation tokens and per-request timing. The `outputs` object returned by `llm.generate()` contains `prompt_tokens`, `generated_tokens`, and (in recent vLLM versions) `ttft` and inter-token latency fields <sup>11</sup>. If these fields are `None`, rely on the benchmark scripts in `vllm/benchmarks` which aggregate per-request metrics.
2. **Record engine metrics after each batch.** Call `llm.get_metrics()` at the end of each eval run to capture metrics such as `gpu_kv_cache_usage` and `prefix_cache_hit_rate` <sup>9</sup>. Log these alongside request-level metrics. When `gpu_kv_cache_usage` is close to 1.0, the engine may swap KV blocks to CPU memory, causing latency spikes. Prefix-cache hit rate quantifies how often the engine reuses previously cached prefixes.
3. **Measure layer-level compute usage.** vLLM wraps the PyTorch model. To see how much each component contributes to latency, modify the model's forward pass (e.g., in a forked copy of the

model class) to wrap each block in a context manager from `torch.autograd.profiler.record_function`. Then run inference and use PyTorch's `torch.profiler` to produce a per-layer time breakdown. For example:

```
import torch
from vllm import LLM, SamplingParams
from torch.profiler import profile, record_function, ProfilerActivity

llm = LLM(model="Qwen/Qwen3.6-35B-A3B", disable_log_stats=False)
prompts = ["def fib(n):", "# Compute factorial"]

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             record_shapes=True) as prof:
    with record_function("batch_generate"):
        outputs = llm.generate(prompts, SamplingParams())

# summarise time per module
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=20))
```

This yields the CUDA time spent in attention kernels, feedforward layers and embedding layers. Because vLLM uses fused kernels (e.g., fused attention/GEMM kernels from Triton or CuTe), the profiler will show them as composite operations. Compare these times across models (8B, 32B, 70B) to identify which components dominate inference.

## Measuring usage at the CuTe-DSL level

CuTe-DSL exposes GPU kernels in Python. To measure how much each component of a model consumes memory and compute at this level:

1. **Use CuTe to implement or inspect model kernels.** For commonly used kernels (GEMM, attention, LayerNorm), you can write equivalent CuTe-DSL kernels or inspect the kernels generated by CuTe in vLLM. For example, the pointer computation formula above arises when computing global memory addresses for tiles <sup>1</sup>.
2. **Measure memory footprint per tile.** Use CuTe's `cute.make_layout` and `cute.make_tensor` to construct the layout for a tensor and call `tensor.shared_memory_bytes()` or `tensor.register_bytes()` to see how much shared memory or registers a tile uses. This allows you to compare different tile shapes or data types.
3. **Instrument kernels with dynamic printing.** Inside CuTe kernels, use `cute.printf` to print dynamic values such as tile indices, pointer addresses and copy operations <sup>12</sup>. This helps verify that the row/column strides are correct and that no bank conflicts occur.
4. **Use Nsight Systems or nvprof for fine-grained profiling.** CuTe kernels compile to CUDA. Use Nsight Systems or Nsight Compute to capture kernel-level metrics—e.g., achieved occupancy, global

memory throughput, shared-memory utilisation and cache hit rates. Compare the performance of the same model component implemented in CuTe vs Triton or PyTorch.

## Measuring KV-cache efficiency across harnesses and models

1. **Set up evaluation harnesses.** Use the `frontier-evals` repository from OpenAI. The `PaperBench` tasks replicate state-of-the-art research papers, `SWE-Lancer` tasks evaluate software-engineering capabilities and `EVMBench` tests smart-contract security. Each eval provides scripts in its `runtime_*` or `scripts/` directories to orchestrate runs <sup>13</sup>.
2. **Use different harnesses and open models.** Start with the `opencode` and `codex` harnesses (provided by Frontier Evals). For each harness, run with several open-source coding models from Hugging Face:
3. **8B class:** use models like `Qwen/Qwen2.5-7B-Instruct` or `DeepSeek-Coder-6.7B` (v4 Flash version) which fit on a single GPU with 8–16 GB of VRAM.
4. **32B class:** use models such as `DeepSeek-Coder-33B` or `Qwen/Qwen2-32B`.
5. **70B class:** use `Llama-2-70B` or `Qwen3.6-35B-A3B` (two MoE experts) depending on VRAM availability.

Ensure that you set appropriate `max_model_len` to avoid over-allocating KV-cache blocks <sup>10</sup>.

1. **Collect KV-cache metrics.** During each eval run, periodically record `vllm:kv_cache_usage_perc` and compute statistics such as mean, max and standard deviation. High values indicate that the KV-cache is nearly full; if it reaches 1.0 the engine may swap blocks to CPU memory. Record `prefix_cache_queries` and `prefix_cache_hits` to compute the prefix-cache hit rate ( $\text{hits} / \text{queries}$ ). Track these metrics separately for the pre-fill phase and decode phase, since pre-fill caches entire prompts while decode caches only new tokens.
2. **Compare harnesses.** The `opencode` harness may use streaming or function-calling patterns that create many short requests, while the `codex` harness may send longer contexts. Analyse how these patterns affect KV-cache usage and prefix-cache hit rate. For example, short streaming requests may result in frequent cache resets and lower hit rates, whereas longer contexts benefit more from prefix reuse.
3. **Analyse throughput and latency.** Use `vllm:e2e_request_latency_seconds`, `vllm:time_to_first_token_seconds` and `vllm:inter_token_latency_seconds` to compute tokens-per-second (TPS) and latency distribution <sup>5</sup>. Compare these across models and harnesses to see how KV-cache pressure influences performance.

## Parsing and analysing logs

1. **Configure logging.** In addition to Prometheus scraping, vLLM writes periodic log entries summarising throughput, KV-cache usage and prefix-cache hit rate. Redirect these logs to a file and parse them using a Python script or `pandas`. Example log line:

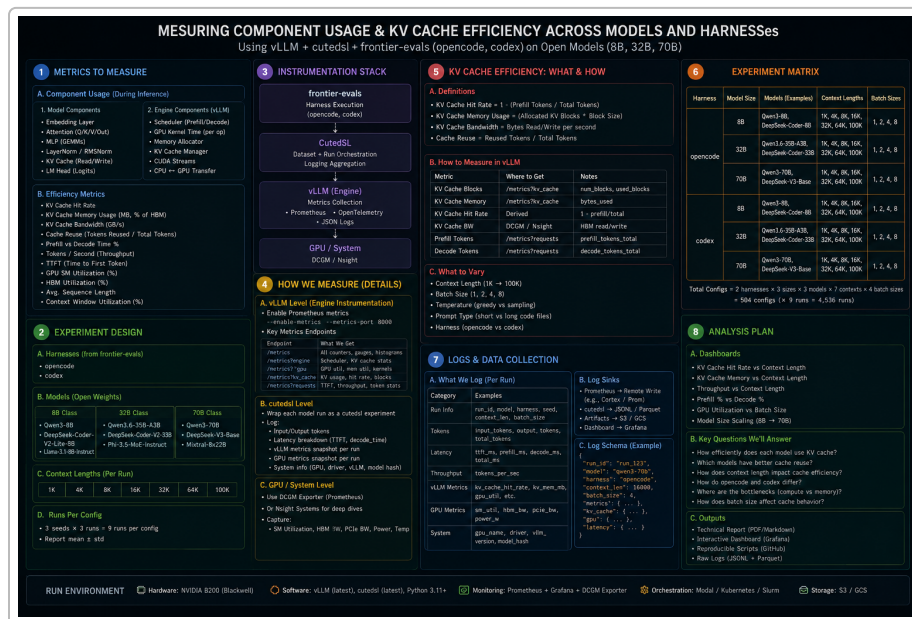
```
INFO stats: requests=120 tokens=4800 time=4.0s ttft=0.08s tpot=0.015s
kv_cache_usage=0.65 prefix_cache_hit_rate=0.12
```

1. **Use JSON for per-request stats.** If you run `benchmark_serving.py` or `throughput.py` with `--save-result`, vLLM will save a JSON file with per-request statistics including TTFT and inter-token latency <sup>14</sup>. Load this JSON to compute histograms, percentiles and correlation with prompt length or harness type.
2. **Post-processing.** For each model and harness, compute summary statistics such as mean KV-cache usage, prefix-cache hit rate, TTFT and TPS. Plot KV-cache usage over time to detect when the cache fills and triggers swapping. Compare per-layer times from PyTorch profiler to determine which components (embedding, attention, feed-forward) contribute most to latency.

## Limitations and considerations

- **MoE models (e.g., Qwen3.6 35B A3B).** Mixture-of-Experts models dynamically select experts per token. vLLM's KV-cache manager allocates separate slots per expert; this can lead to imbalanced usage. At the time of writing, metrics for MoE KV-cache residency are limited and may require inspecting vLLM source code or instrumenting the `KvCacheManager`.
- **Per-component usage in fused kernels.** vLLM fuses several operations (e.g., rope rotation + attention). Profiling at the vLLM level will not separate these operations. To isolate them, run the same model in raw PyTorch or with CuTe kernels and profile each function.
- **Hardware counters.** For absolute measurements of memory bandwidth, occupancy and cache utilisation, use NVIDIA's Nsight Compute on each kernel. This gives deeper insight than vLLM's high-level metrics but requires root access and may slow the evaluation.

## Visualising the GPU memory hierarchy (optional)



The pyramid illustrates the memory sizes and bandwidths of registers, shared memory (~220 kB per SM with ~20 TB/s bandwidth), L2 cache, and HBM (tens of GB with hundreds of GB/s bandwidth). Efficient kernel design moves data from HBM to shared memory in tiles and ensures that each warp accesses contiguous memory, as expressed by the pointer formula <sup>1</sup>.

## Summary

To measure how much each component of a model is used in inference and to evaluate KV-cache efficiency across harnesses and models, instrument both the vLLM engine and the underlying model. Use vLLM's built-in metrics ( `kv_cache_usage_perc`, `prefix_cache_hits`, latency histograms) and per-request data. Collect these metrics while running `frontier-evals` tasks with `opencode` and `codex` harnesses on a variety of open models. Supplement with PyTorch profiling to attribute compute time to individual layers. At the CuTe-DSL level, inspect kernel layouts and memory usage through its layout primitives and measure kernel performance with Nsight tools. Finally, parse the logs and metrics to compare how harness patterns and model sizes affect KV-cache usage, throughput, and latency.

<sup>1</sup> <sup>12</sup> CuTe DSL Basics: A Practical Introduction — Chris Choy

<https://chrischoy.org/posts/cutedsl-basics/>

<sup>2</sup> <sup>3</sup> <sup>4</sup> <sup>5</sup> <sup>6</sup> Metrics - vLLM

<https://docs.vllm.ai/en/stable/design/metrics/>

<sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>11</sup> <sup>14</sup> How to get the log for benchmarking - Benchmarking - vLLM Forums

<https://discuss.vllm.ai/t/how-to-get-the-log-for-benchmarking/2235>

10 Mastering vLLM KV-Cache: 10 Battle-Tested Tweaks for Maximum Token Throughput | by TechEon | Medium

<https://atul4u.medium.com/mastering-vllm-kv-cache-10-battle-tested-tweaks-for-maximum-token-throughput-9101a4917c5a>

13 GitHub - openai/frontier-evals: OpenAI Frontier Evals · GitHub

<https://github.com/openai/frontier-evals>